Assignment No:1-A

Title: To use PCA Algorithm for dimensionality reduction.
You have a dataset that includes measurements for different variables on wine (alcohol, ash, magnesium, and so on). Apply PCA algorithm & transform this data so that most variations in the measurements of the variables are captured by a small number of principal components so that it is easier to distinguish between red and white wine by inspecting these principal components.

Dataset Link: https://media.geeksforgeeks.org/wp-content/uploads/Wine.csv

Objectives: To make use of PCA algorithm
To transform the data in reduced form

Theory:

Principal Component Analysis is an unsupervised learning algorithm that is used for the dimensionality reduction in machine learning. It is a statistical process that converts the observations of correlated features into a set of linearly uncorrelated features with the help of orthogonal transformation. These new transformed features are called the **Principal Components**. It is one of the popular tools that is used for exploratory data analysis and predictive modeling.

It is a technique to draw strong patterns from the given dataset by reducing the variances. The PCA algorithm is based on some mathematical concepts such as:

- Variance and Covariance
- Eigenvalues and Eigen factors

Some common terms used in PCA algorithm:

- **Dimensionality:** It is the number of features or variables present in the given dataset. More easily, it is the number of columns present in the dataset.
- **Correlation:** It signifies how strongly two variables are related to each other. Such as if one changes, the other variable also gets changed. The correlation value ranges from -1 to +1. Here, -1 occurs if variables are inversely proportional to each other, and +1 indicates that variables are directly proportional to each other.
- **Orthogonal:** It defines that variables are not correlated to each other, and hence the correlation between the pair of variables is zero.
- **Eigenvectors:** If there is a square matrix M, and a non-zero vector v is given. Then v will be eigenvector if Av is the scalar multiple of v.

- **Covariance Matrix:** A matrix containing the covariance between the pair of variables is called the Covariance Matrix.

Steps for PCA algorithm

1. **Getting the dataset**
   Firstly, we need to take the input dataset and divide it into two subparts X and Y, where X is the training set, and Y is the validation set.
2. **Representing data into a structure**
   Now we will represent our dataset into a structure. Such as we will represent the two-dimensional matrix of independent variable X. Here each row corresponds to the data items, and the column corresponds to the Features. The number of columns is the dimensions of the dataset.
3. **Standardizing the data**
   In this step, we will standardize our dataset. Such as in a particular column, the features with high variance are more important compared to the features with lower variance.
   If the importance of features is independent of the variance of the feature, then we will divide each data item in a column with the standard deviation of the column. Here we will name the matrix as Z.
4. **Calculating the Covariance of Z**
   To calculate the covariance of Z, we will take the matrix Z, and will transpose it. After transpose, we will multiply it by Z. The output matrix will be the Covariance matrix of Z.
5. **Calculating the Eigen Values and Eigen Vectors**
   Now we need to calculate the eigenvalues and eigenvectors for the resultant covariance matrix Z. Eigenvectors or the covariance matrix are the directions of the axes with high information. And the coefficients of these eigenvectors are defined as the eigenvalues.
6. **Sorting the Eigen Vectors**
   In this step, we will take all the eigenvalues and will sort them in decreasing order, which means from largest to smallest. And simultaneously sort the eigenvectors accordingly in matrix P of eigenvalues. The resultant matrix will be named as P*.
7. **Calculating the new features Or Principal Components**
   Here we will calculate the new features. To do this, we will multiply the P* matrix to the Z.

In the resultant matrix Z*, each observation is the linear combination of original features. Each column of the Z* matrix is independent of each other.

8. **Remove less or unimportant features from the new dataset.**

   The new feature set has occurred, so we will decide here what to keep and what to remove. It means, we will only keep the relevant or important features in the new dataset, and unimportant features will be removed out.

Conclusion:

The performance of logistic regression model is improved after performing principal component analysis. PCA not only removed some redundancy but also improved variance in the dataset

Reference:

https://www.kaggle.com/code/bhavesh302/pca-on-wine-dataset

Assignment No:2A

Title: Implement Linear Regression Using Python

Dataset : Uber Dataset

Objectives: To Apply different regression techniques for making predictions in different applications.

Theory:

Regression: Regression analysis is one of the most important fields in statistics and machine learning.There are many regression methods available. Linear regression is one of them.

What Is Regression?

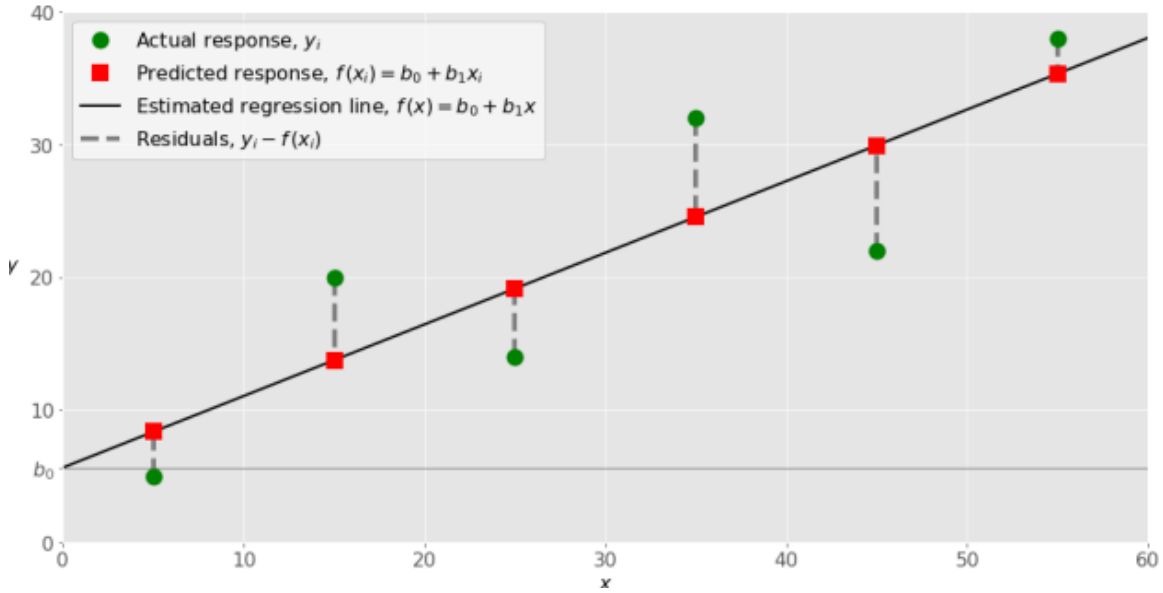
Regression analysis is one of the most important fields in statistics and machine learning. There Are many regression methods available. Linear regression is one of them. Regression searches for relationships among variables.

For example, you can observe several employees of some company and try to understand how their salaries depend on the features, such as experience, level of education, role, city they work in, and so on.This is a regression problem where data related to each employee represent one observation.

The presumption is that the experience, education, role, and city are the independent features, while the salary depends on them. Generally, in regression analysis, you usually consider some phenomenon of interest and There have a number of observations. Each observation has two or more features. Following the assumption that(atleast)one of the features depends on the others,you try to establish relation among them. you need to find a function that maps some features or variables to others sufficiently well. The dependent features are called the dependent variables, outputs, or responses. The independent features are called the independent variables, inputs, or predictors.

Linear Regression: Linear regression is probably one of the most important and widely used regression techniques. It's among the simplest regression methods. One of its main advantages is the ease of interpreting results .When implementing linear regression of some dependent variable □on the set of independent variables x= ($x1 \ldots xr_r$), where r is the number of predictors, you assume a linear relationship between y and x: $y= \beta0 + \beta1x1 + \ldots\ldots+ \beta rxr+ \epsilon$. This equation is the regression equation. $\beta0$,, βr_r are there regression coefficients, and □is the random error. Linear regression calculates the estimators of the regression coefficients or simply the predicted weights, denoted with b1…br. They define the estimated regression function f(x) = b0 +b1x1 +….+brxr This function should capture the dependencies between the input and output sufficiently well.

Simple Linear Regression: The following figure illustrates simple linear regression:



When implementing simple linear regression, you typically start with a given set of input-output (x-y) pairs (green circles). These pairs are your observations. For example, the leftmost observation (green circle) has the input x= 5 and the actual output (response) y= 5. The next one has x= 15 and y= 20, and so on. The Estimated Regression Function(blackline)has the equation f(x)=b0+b1x.Your goal is to calculate the optimal values of the predicted weights b0 and b1 that minimize SSR and determine the estimated regression function.Thevalueofb0,also called the intercept, shows the point where the estimated regression line crosses the y axis. It's the value Of The Estimated Response f(x)for x=0. The Value of b1 determines the slope of the estimated regression line. The predicted responses (red squares) are the points on the regression line that correspond to the input values. For example, for the input x= 5, the predicted response is f (5) = 8.33 (represented with the leftmost red square). Theresiduals(vertical dashed gray lines) can be calculated as $y1_i$-f($x1_i$)=y1–b0–b1x1 for=1, …,n . They are the distances between the green circles and red squares. When you implement linear regression, you are actually trying to minimize these distances and make the red squares as close to the predefined green circles as possible.

Implementing Linear Regression in Python It's time to start implementing linear regression in Python. Basically, all you should do is apply the proper packages and their functions and classes. Python Packages for Linear Regression The package NumPy is a fundamental Python scientific package that allows many high-performance operations on single-and multidimensional arrays. It also offers many Mathematical routines. Of course, it's open source. The package scikit-learn is a widely used Python library for machine learning, built on top of NumPy and some other packages. It provides the means for preprocessing data, reducing dimensionality, implementing regression, classification, clustering, and more .Like NumPy, scikit learn is also open source. If you want to implement linear regression and need the functionality beyond the scope of scikit learn, you should consider stats models. It's a powerful Python package for the estimation of statistical models, performing tests, and more. It's open source as well. Simple Linear Regression With scikit-learn Let's start with the simplest case, which is simple linear regression.

There are five basic steps when you're implementing linear regression:

a. Import the packages and classes you need.
b. Provide data to work with and eventually do appropriate transformations.
c. Create a regression model and fit it with existing data.
d. Check the results of model fitting to know whether the model is satisfactory.
e. Apply the model for predictions.

SOURCE CODE:
Refer to link below for more information

https://www.kaggle.com/code/mrgeislinger/simple-linear-regression-example-w-scikit-learn

Conclusion:

Thus implementation of Linear Regression Algorithm is done successfully.

Assignment No:3A

Title: Implementation of Support Vector Machines (SVM) for classifying images of hand-written digits into their respective numerical classes (0 to 9).

Objectives: Apply the classification algorithms to classify the data with appropriate labels.

Theory:

Support Vector Machines (SVMs) are a type of supervised machine learning algorithm that can be used for classification and regression tasks. In this article, we will focus on using SVMs for image classification.

When a computer processes an image, it perceives it as a two-dimensional array of pixels. The size of the array corresponds to the resolution of the image, for example, if the image is 200 pixels wide and 200 pixels tall, the array will have the dimensions 200 x 200 x 3. The first two dimensions represent the width and height of the image, respectively, while the third dimension represents the RGB color channels. The values in the array can range from 0 to 255, which indicates the intensity of the pixel at each point.

In order to classify an image using an SVM, we first need to extract features from the image. These features can be the color values of the pixels, edge detection, or even the textures present in the image. Once the features are extracted, we can use them as input for the SVM algorithm.

The SVM algorithm works by finding the hyperplane that separates the different classes in the feature space. The key idea behind SVMs is to find the hyperplane that maximizes the margin, which is the distance between the closest points of the different classes. The points that are closest to the hyperplane are called support vectors.

One of the main advantages of using SVMs for image classification is that they can effectively handle high-dimensional data, such as images. Additionally, SVMs are less prone to overfitting than other algorithms such as neural networks.

In machine learning where the model is trained by input data and expected output data.

Algorithm:

SVM-based Handwritten Digit Classification

1. Load the handwritten digit dataset.
2. Preprocess and normalize the input images.
3. Flatten each image into a feature vector.
4. Split the dataset into training and testing data.
5. Create an SVM classifier with a suitable kernel.
6. Train the SVM using the training dataset.
7. Predict digit classes for the test dataset.
8. Calculate classification accuracy and generate a confusion matrix.
9. Display the predicted classes.
10. Stop.

Conclusion:

Students can create and train a Support Vector Machine (SVM) model to accurately classify images of cats and dogs. The best parameters for the SVM model were determined using Grid Search CV, and the model's accuracy was measured.

Assignment No:4A

Title: Clustering Analysis

Objectives: To find optimum number of clusters using elbow method
To apply K-Means Algorithm on Iris dataset

Theory:

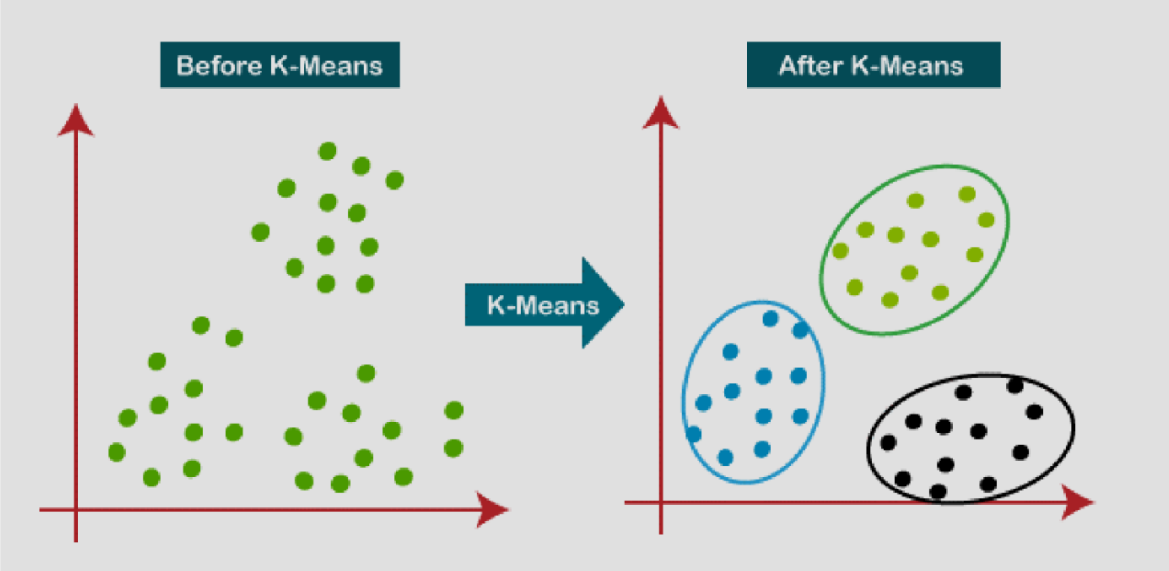
Clustering is a technique in which the data points are arranged in similar groups dynamically without any pre-assignment of groups. For example, here is a simple plot of data points. As you can see, some set of points are closer to each other when compared with others. These sets could possibly form a group (or a cluster) for further data analytics K-Means Clustering K-Means Clustering is an unsupervised learning algorithm that is used to solve the clustering problems in machine learning or data science.K-Means Clustering is an Unsupervised Learning algorithm, which groups the unlabeled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if K=2, there will be two clusters, and for K=3, there will be three clusters, and so on.

It is an iterative algorithm that divides the unlabeled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties. It allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabeled dataset on its own without the need for any training.

It is a centroid based algorithm, where each cluster is associated with a centroid. The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters. The algorithm takes the unlabeled dataset as input, divides the dataset into k-number of clusters, and repeats the process until it does not find the best clusters. The value of k should be predetermined in this algorithm. The k-means clustering algorithm mainly performs two tasks:

- Determines the best value for K center points or centroids by an iterative process.

- Assigns each data point to its closest k-center. Those data points which are near to the k-center, create a cluster. The below diagram explains the working of the K-means Clustering Algorithm:



The working of the K-Means algorithm is explained in the below steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.

Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.

Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

Step-7: The model is ready.

Algorithm:

At a high-level, the following steps are taken for clustering the data using K-means clustering algorithm.

1. Decide the number of clusters, k , that you desire to group your data points into.

2. Select k random data points as centroids.

3. Compute the distance from each data point to each centroid. Assign all the data points to the closest cluster centroid. The distance d between any two points, (x1, y1) and (x2, y2) is calculated as

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

4. Recompute the centroids of newly formed clusters. The centroid (Xc, Yc) of the m data points in a cluster is calculated as

$$(X_c, Y_c) = \left(\frac{\sum_{i=1}^{m} X_i}{m}, \frac{\sum_{i=1}^{m} Y_i}{m} \right)$$

It is a simple arithmetic mean of all X coordinates and Y coordinates of the m data points in the cluster.

5. Repeat steps 3 and 4 until any of the following criteria is met

a. Centroids of newly formed clusters do not change.

b. Points remain in the same cluster.

c. Maximum number of iterations are reached as desired.

Elbow Method

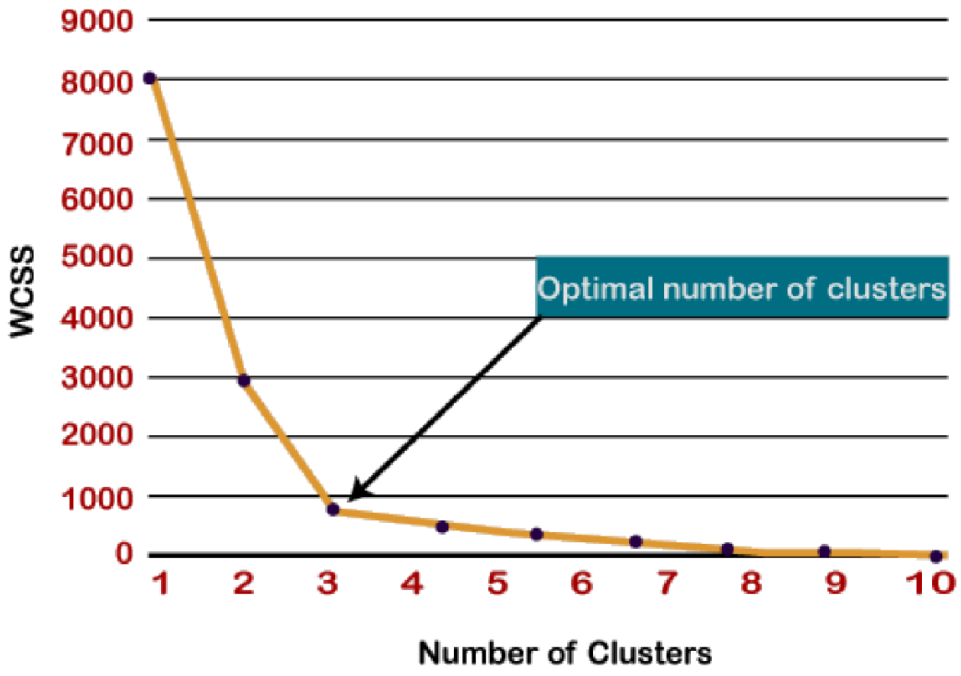
The Elbow method is one of the most popular ways to find the optimal number of clusters. This method uses the concept of WCSS value. WCSS stands for Within Cluster Sum of Squares, which defines the total variations within a cluster. The formula to calculate the value of WCSS (for 3 clusters) is given below:

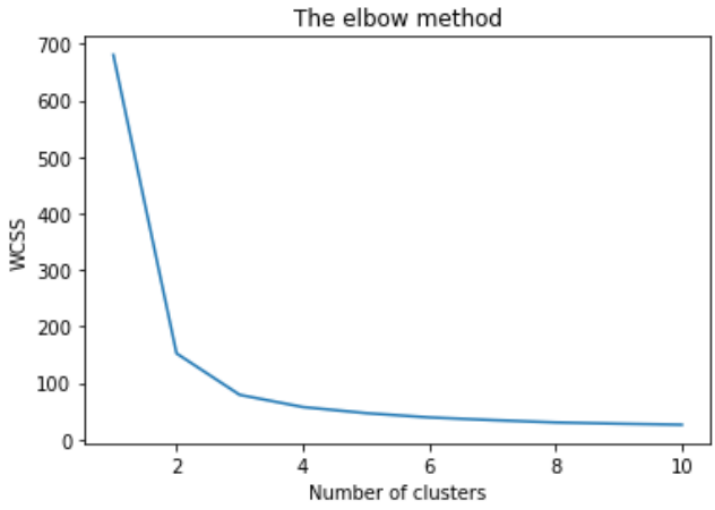
$$WCSS = \sum_{P_i \text{ in Cluster1}} distance(P_i\ C_1)^2 + \sum_{P_i \text{ in Cluster2}} distance(P_i\ C_2)^2 + \sum_{P_i \text{ in CLuster3}} distance(P_i\ C_3)^2$$

In the above formula of WCSS, ΣPi in Cluster1 distance (Pi C1)2: It is the sum of the square of the distances between each data point and its centroid within a cluster1 and the same for the

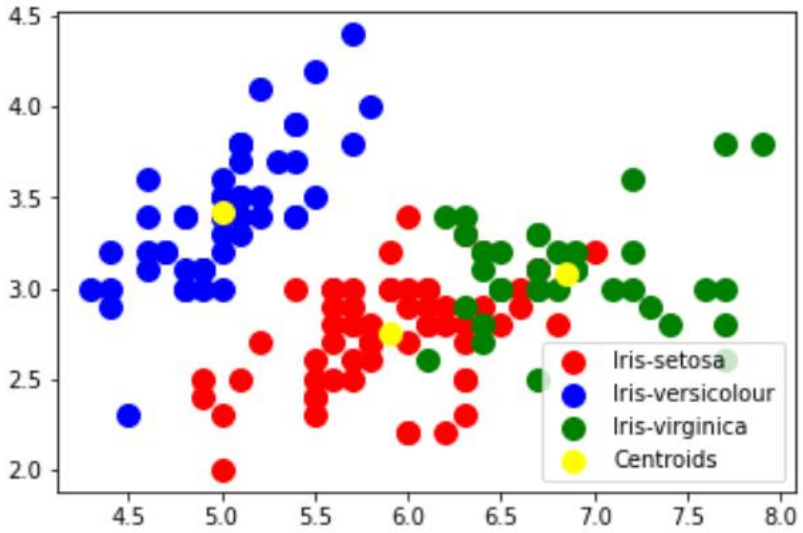
other two terms. To measure the distance between data points and centroid, we can use any method such as Euclidean distance or Manhattan distance. To find the optimal value of clusters, the elbow method follows the below steps:

- It executes the K-means clustering on a given dataset for different K values (ranges from 1-10).
- For each value of K, calculates the WCSS value.
- Plots a curve between calculated WCSS values and the number of clusters K.
- The sharp point of bend or a point of the plot looks like an arm, then that point is considered as the best value of K. Since the graph shows the sharp bend, which looks like an elbow, hence it is known as the elbow method. The graph for the elbow method looks like the below image:





You can clearly see why it is called 'The elbow method' from the above graph, the optimum clusters is where the elbow occurs. This is when the within cluster sum of squares (WCSS) doesn't decrease significantly with every iteration. Now that we have the optimum amount of clusters, we can move on to applying K-means clustering to the Iris dataset.



Conclusion:

Students will be able to apply K-Means Algorithm & Elbow method on Dataset to find Optimum no of Clusters

Assignment No:5-A

Title: Ensemble Learning

Implement Random Forest Classifier model to predict the safety of the car.

Dataset Link: https://www.kaggle.com/datasets/elikplim/car-evaluation-data-set

Objectives: To apply Random Forest Classifier & build model for prediction

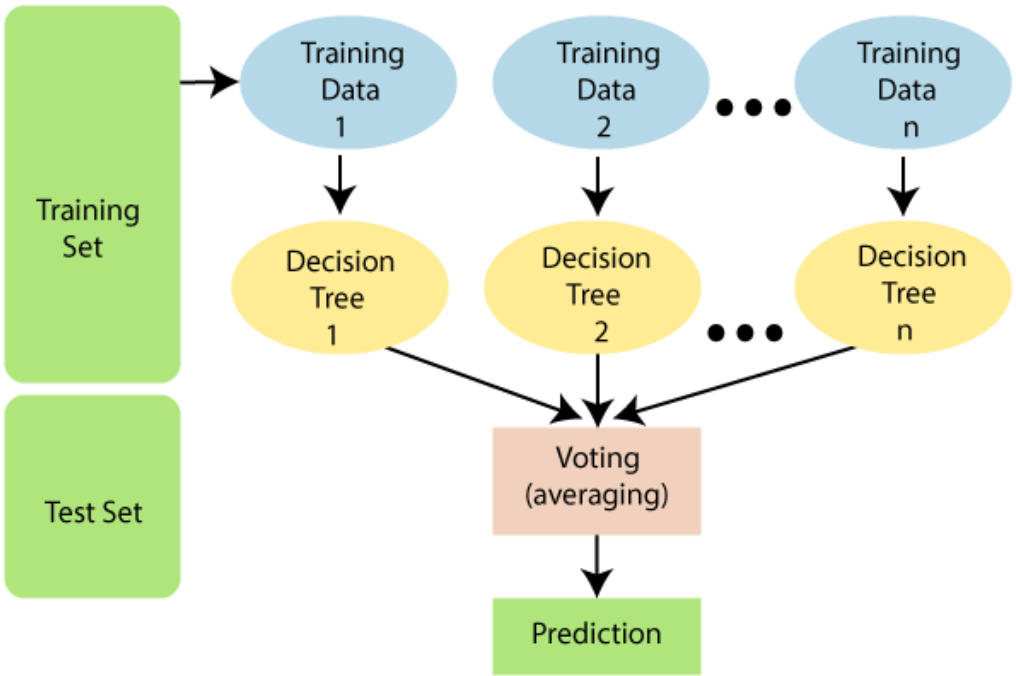
Theory:

Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique. It can be used for both Classification and Regression problems in ML. It is based on the concept of ensemble learning, which is a process of combining multiple classifiers to solve a complex problem and to improve the performance of the model.

Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset. Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.

The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

The below diagram explains the working of the Random Forest algorithm



Random forest algorithm intuition can be divided into two stages.

In the first stage, we randomly select “k” features out of total m features and build the random forest. In the first stage, we proceed as follows:

1. Randomly select k features from a total of m features where k < m.
2. Among the k features, calculate the node d using the best split point.
3. Split the node into daughter nodes using the best split.
4. Repeat 1 to 3 steps until l number of nodes has been reached.
5. Build forest by repeating steps 1 to 4 for n number of times to create n number of trees.

In the second stage, we make predictions using the trained random forest algorithm.

1. We take the test features and use the rules of each randomly created decision tree to predict the outcome and stores the predicted outcome.
2. Then, we calculate the votes for each predicted target.
3. Finally, we consider the high voted predicted target as the final prediction from the random forest algorithm.

Working of Random Forest Algorithm

Random Forest works in two-phase first is to create the random forest by combining N decision tree, and second is to make predictions for each tree created in the first phase.

The Working process can be explained in the below steps and diagram:

Step-1: Select random K data points from the training set.

Step-2: Build the decision trees associated with the selected data points (Subsets).

Step-3: Choose the number N for decision trees that you want to build.

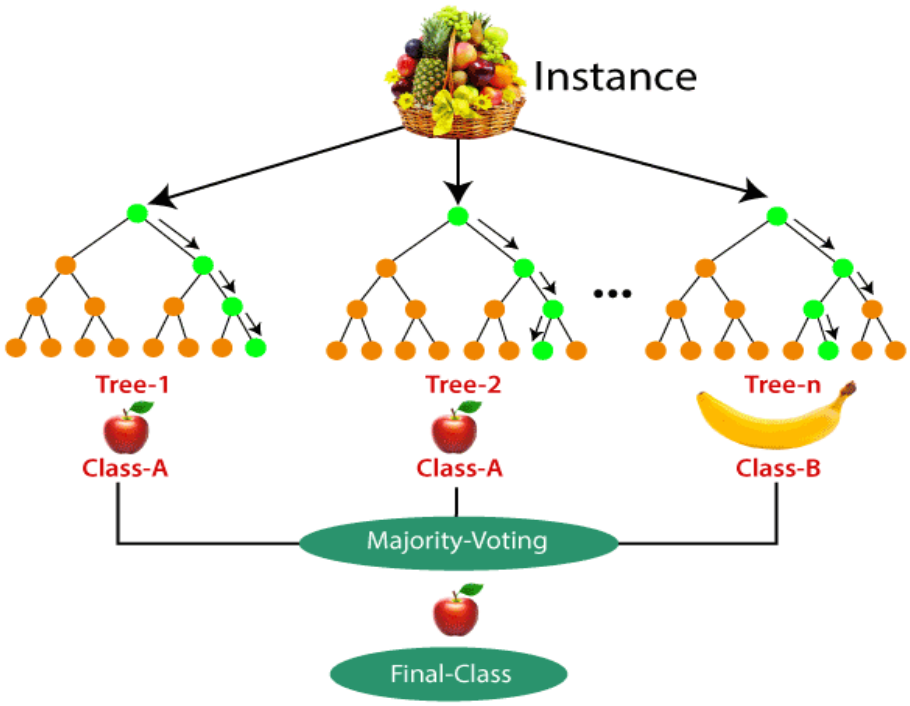
Step-4: Repeat Step 1 & 2.

Step-5: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

The working of the algorithm can be better understood by the below example:

Example: Suppose there is a dataset that contains multiple fruit images. So, this dataset is given to the Random forest classifier. The dataset is divided into subsets and given to each decision tree. During the training phase, each decision tree produces a prediction result, and when a new data point occurs, then

based on the majority of results, the Random Forest classifier predicts the final decision. Consider the below image:



Advantages and disadvantages of Random Forest algorithm

The advantages of Random forest algorithm are as follows:

1. Random forest algorithm can be used to solve both classification and regression problems.
2. It is considered as very accurate and robust model because it uses large number of decision-trees to make predictions.
3. Random forests takes the average of all the predictions made by the decision-trees, which cancels out the biases. So, it does not suffer from the overfitting problem.
4. Random forest classifier can handle the missing values. There are two ways to handle the missing values. First is to use median values to replace continuous variables and second is to compute the proximity-weighted average of missing values.
5. Random forest classifier can be used for feature selection. It means selecting the most important features out of the available features from the training dataset.

The disadvantages of Random Forest algorithm are listed below:

1. The biggest disadvantage of random forests is its computational complexity. Random forests is very slow in making predictions because large number of decision-trees are used to make

predictions. All the trees in the forest have to make a prediction for the same input and then col_names = ['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'class']

2. df.columns=col_names

3. perform voting on it. So, it is a time-consuming process.
4. The model is difficult to interpret as compared to a decision-tree, where we can easily make a prediction as compared to a decision-tree.

Algorithm – Random Forest Classifier

1. **Start**
2. Load the car safety dataset.
3. Preprocess and encode categorical data.
4. Separate input features and target safety class.
5. Split data into training and testing sets.
6. Create and train the **Random Forest Classifier**.
7. Predict car safety using the test data.
8. Evaluate the model using **accuracy and confusion matrix**.
9. Display the results.
10. **Stop**.

Conclusion:

Students will be able to apply Random Forest Classifier to perform prediction on dataset.

Assignment 6 – A : Reinforcement Learning

A. Implement Reinforcement Learning using an example of a maze environment that the agent needs to explore.

Title: Reinforcement Learning: Maze Exploration

Introduction:
we will explore the concept of reinforcement learning by implementing an agent that learns to navigate and explore a maze environment. The agent will use a popular algorithm called Q-learning to learn an optimal policy for reaching the goal state while avoiding obstacles.

Prerequisites:

Basic knowledge of Python programming language.
Familiarity with the concept of reinforcement learning.

Requirements:
Python 3.x
NumPy library

Lab Steps:

Step 1: Setting up the Maze Environment
Create a grid-based maze environment with walls, start position, and goal position. Represent the maze as a 2D grid, where each cell can be either empty, a wall, the start position, or the goal position.

Step 2: Implementing the Agent

1. Initialize the Q-table as a 2D array with dimensions equal to the size of the maze grid. Set all Q-values to zero initially.

2. Implement the Q-learning algorithm:
   a. Choose an action (movement direction) for the agent based on the current state using an epsilon-greedy policy. This policy balances exploration (random actions) and exploitation (choosing the action with the highest Q-value).
   b. Take the chosen action and observe the next state and reward.
   c. Update the Q-value for the previous state-action pair using the Q-learning update equation:
   Q[state][action] = Q[state][action] + learning_rate * (reward + discount_factor * max(Q[next_state]) - Q[state][action])
   d. Repeat steps a-c until the agent reaches the goal state or a maximum number of iterations is reached.
   e. Update the current state to the next state and continue with the next iteration.

Step 3: Training the Agent

1. Initialize the necessary hyperparameters:
    - Epsilon (exploration rate): controls the probability of choosing random actions.
    - Learning rate: controls the weight given to the new information compared to the existing Q-values.
    - Discount factor: determines the importance of future rewards.
    - Maximum number of episodes: defines the number of training iterations.
    - Maximum number of steps per episode: limits the number of steps the agent can take to reach the goal.
2. Run multiple episodes of the training process, repeating steps 2a-2e for each episode.
3. Decrease the exploration rate (epsilon) over time to gradually shift from exploration to exploitation.
4. Monitor the agent's progress by tracking metrics such as the total rewards obtained and the number of steps taken per episode.

Step 4: Testing the Agent

1. After training, test the agent's learned policy by letting it navigate the maze.
2. Start the agent at the start position and follow the learned policy until it reaches the goal or maximum steps are reached.
3. Record the agent's performance metrics, such as the number of steps taken and the success rate of reaching the goal.

Conclusion:

In this lab, we implemented a reinforcement learning agent that learned to navigate a maze environment using the Q-learning algorithm. By iteratively exploring and updating the Q-values, the agent gradually discovered an optimal policy for reaching the goal while avoiding obstacles. This exercise provides a hands-on understanding of reinforcement learning concepts and how they can be applied to real-world problems.

Experiment Execution Code:

```
import numpy as np

# Define the maze environment
maze = np.array([
    [0, 0, 0, 0, 0],
    [0, -1, -1, -1, 0],
    [0, 0, 0, 0, 0],
    [0, -1, -1, -1, 0],
    [0, 0, 0, 0, 0],
])
start_state = (0, 0)
goal_state = (4, 4)

# Define hyperparameters
epsilon = 0.1
```

```
learning_rate = 0.5
discount_factor = 0.9
max_episodes = 1000
max_steps = 100

# Initialize the Q-table
num_rows, num_cols = maze.shape
num_actions = 4  # 4 possible actions: up, down, left, right
Q = np.zeros((num_rows, num_cols, num_actions))

# Q-learning algorithm
for episode in range(max_episodes):
    state = start_state
    total_reward = 0

    for step in range(max_steps):
        # Choose an action using epsilon-greedy policy
        if np.random.uniform(0, 1) < epsilon:
            action = np.random.randint(num_actions)
        else:
            action = np.argmax(Q[state])

        # Take the chosen action and observe the next state and reward
        if action == 0:  # up
            next_state = (state[0] - 1, state[1])
        elif action == 1:  # down
            next_state = (state[0] + 1, state[1])
        elif action == 2:  # left
            next_state = (state[0], state[1] - 1)
        elif action == 3:  # right
            next_state = (state[0], state[1] + 1)

        # Check if the next state is valid
        if next_state[0] >= 0 and next_state[0] < num_rows and next_state[1] >= 0 and
next_state[1] < num_cols:
            reward = maze[next_state]
        else:
            reward = -5  # penalize for hitting walls or going out of bounds

        # Update the Q-value for the previous state-action pair
        Q[state][action] = Q[state][action] + learning_rate * (
              reward + discount_factor * np.max(Q[next_state]) - Q[state][action]
        )

        # Update the current state
        state = next_state
```